

## Generative AI Project using IBM Cloud – HEALTHAI

### Project Documentation Format

- **Introduction**

- **Project Title: HEALTHAI: Intelligent Healthcare Assistant using IBM Granite (Generative AI with IBM Cloud)** □ **Team Members:**

- **Pulivarthi Balasubrahmanyam (Team Leader – Requirements gathering ,Interaction& Testing):**

Contributed by assisting in prompt design, testing the AI model outputs across modules like Disease Prediction and Health Chat, and refining interactions with IBM Granite.

- **Neela Deeven Paul (Team Member - Development , Integration , Deployment & documentation):**

Led the complete development of the HEALTHAI application, including IBMGranite integration, Streamlit-based UI design, module creation, and model API handling,project deployment

- **Movva Vamsi Krishna (Team Member - UI Structuring , Feature Enhancement & Testing):**

Supported in designing user flow, organizing the Streamlit interface across all modules, and suggesting improvements in user interaction and feature behavior.

- **Nallam Setty Mounish Royal (Team Member - Feature Enhancement ,Testing & documentation):**

Supported in- Feature Enhancement of project, testing the project and helped in the documentation.

- **Project Overview**

- **Purpose:**

To build a Generative AI-based healthcare assistant using IBM Granite, capable of answering health queries, predicting diseases, suggesting treatments, and displaying analytics.

- **Features:**

-  AI Health Chat using IBM Granite
- ○ □ Disease Prediction from user symptoms

- 🕒 Treatment Plan Suggestions

D Health Analytics Dashboard

- ☐ Centralized shared model for performance optimization

- **Architecture**

- **Frontend:**

- Built using **Streamlit** for a clean and responsive web interface. Each feature is modularized for easy navigation via sidebar.

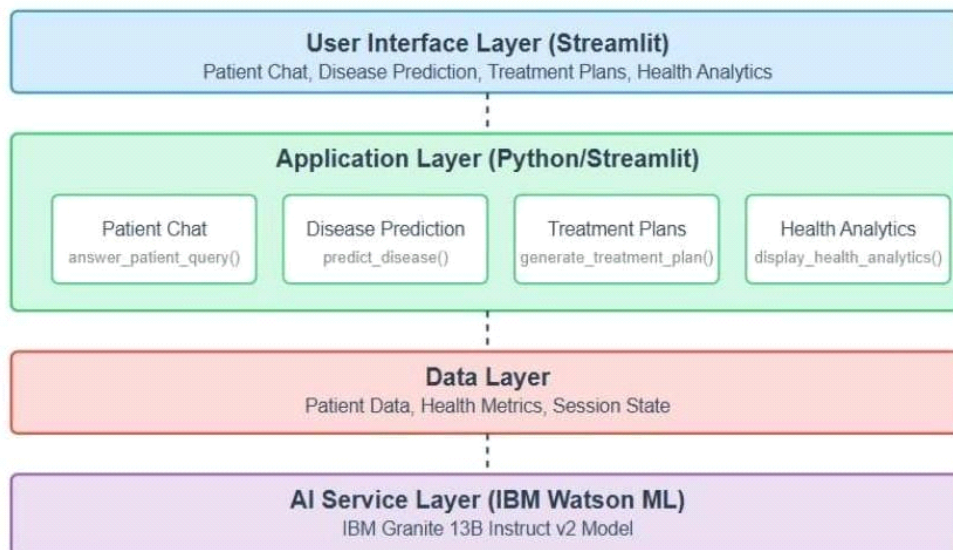
- **Backend & Model:**

- No traditional backend. All logic handled in Streamlit using Python.
    - Uses **IBM Granite 3-2b Instruct model** from IBM Watsonx: ibm-granite/granite-3.2b-instruct
    - Supports both API and **local model loading** (granite/folder).

- **Shared Model Loader:**

- The shared\_model.py file centrally loads and shares the AI model across modules to prevent memory crashes and redundancy.

### HealthAI - Architecture Diagram



- **Setup Instructions**

### **Prerequisites**

- Python 3.10+
- pip
- IBM cloud account and Streamlit Community Cloud account
- Installed model files if using local (granite/ folder) **Installation**

git clone : [https://github.com/deeven17/HealthCare\\_AI](https://github.com/deeven17/HealthCare_AI)

cd Healthai :

pip install -r requirements.txt

### **Environment Variables**

Create a .env file in the root folder:

IBM watsonx\_API\_Key= ESdPliW78JY8LM32Vp6ujw\_EVP1dySvqb5ZQ5lc7K4wi

✓ .env file must be excluded in .gitignore.

- **Folder Structure**

Health-ai/

|                         |                                    |
|-------------------------|------------------------------------|
| — app.py                | # Main entry point                 |
| — shared_model.py       | # Shared AI model instance         |
| — patient_chat.py       | # AI Health Chat module            |
| — disease_prediction.py | # Disease Prediction logic         |
| — treatment_plans.py    | # Treatment Plan suggestions       |
| — health_analytics.py   | # Analytics module                 |
| — requirements.txt      | # Python dependencies              |
| — .env                  | # API token (not pushed to GitHub) |
| — granite/              | # [Optional] Local model folder    |

└─ assets/                    # Logos and screenshots

- **Running the Application**

**For IBM watsonx API:**

streamlit run app.py

**For Local Model:**

Ensure granite/ folder contains the downloaded model and tokenizer files. In  
shared\_model.py, update: model\_path = "./granite"

- **API Documentation**

**Endpoint:** <https://eu-de.ml.cloud.ibm.com/ml/v1/text/chat?version=2023-05-29>

**Headers:**

```
{  
  "Authorization": "Bearer <IBM_API_Key>",  
  "Content-Type": "application/json"  
}
```

**Example Request:**

```
{  
  "inputs": "What are the symptoms of diabetes?"  
}
```

**Example Response:**

```
{  
  "generated_text": "Common symptoms of diabetes include frequent urination..."  
}
```

- **Authentication**

- Streamlit cloud Secrets is securely stored the .env credentials.
- .env is excluded via .gitignore

- App is currently public and stateless (no user login)
- HuggingFace or Firebase Auth can be added in future

- **User Interface**

- Built entirely with **Streamlit**
- Sidebar for navigation
- Text/chat inputs for interaction
- Visual graphs and health tips in Analytics
- Centralized theme and branding

- **Testing**

- ✓ Manual testing across all modules
- ✓ Model tested with varied prompts and edge cases
- ✓ Handled errors for invalid inputs and model timeouts

```

1  # Importing necessary libraries
2  import requests
3  import json
4  import random
5
6  # Base URL for the cloud API
7  cloud_api = "https://cloud.the.com/identity/token"
8
9  # Headers for the cloud API
10 headers = {
11     "Content-Type": "application/json",
12     "Authorization": "Bearer token"
13 }
14
15 # Function to get an access token
16 def get_access_token():
17     url = cloud_api
18     headers = headers
19     data = {
20         "grant_type": "refresh_token",
21         "refresh_token": "your_refresh_token_here"
22     }
23     response = requests.post(url, headers=headers, data=data)
24     if response.status_code == 200:
25         token = response.json().get("access_token")
26         print("Access token received")
27         return token
28     else:
29         print("Error fetching access token")
30         return None
31
32 # Function to get a response from the model
33 def get_response(prompt):
34     url = "https://api.openai.com/v1/completions"
35     headers = {
36         "Content-Type": "application/json",
37         "Authorization": "Bearer token"
38     }
39     data = {
40         "model": "gpt-3.5-turbo",
41         "prompt": prompt,
42         "max_tokens": 150
43     }
44     response = requests.post(url, headers=headers, data=json.dumps(data))
45     if response.status_code == 200:
46         result = response.json().get("choices")[0].get("text")
47         print("Response received")
48         return result
49     else:
50         print("Error fetching response")
51         return None
52
53 # Function to predict diseases from symptoms
54 def predict_disease(symptoms):
55     prompt = "A patient is experiencing the following symptoms: {symptoms}. Please consider the most common and likely causes first, based on global clinical guidelines. Avoid listing rare or severe diseases unless strongly indicated. List top 3 possible diagnoses with estimated likelihoods in percentage format. Example:"
56     response = get_response(prompt)
57     return response
58
59 # Main function
60 if __name__ == "__main__":
61     token = get_access_token()
62     if token:
63         prompt = "A patient is experiencing the following symptoms: {symptoms}. Please consider the most common and likely causes first, based on global clinical guidelines. Avoid listing rare or severe diseases unless strongly indicated. List top 3 possible diagnoses with estimated likelihoods in percentage format. Example:"
64         response = get_response(prompt)
65         print(response)
66     else:
67         print("Error fetching access token")
68

```

- **Screenshots or Demo**



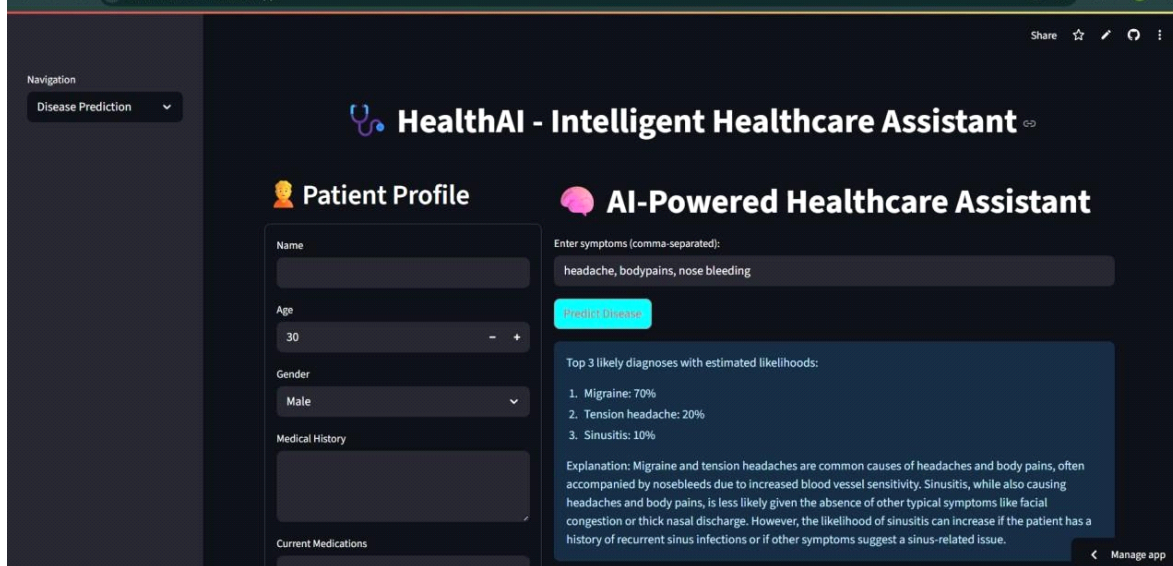
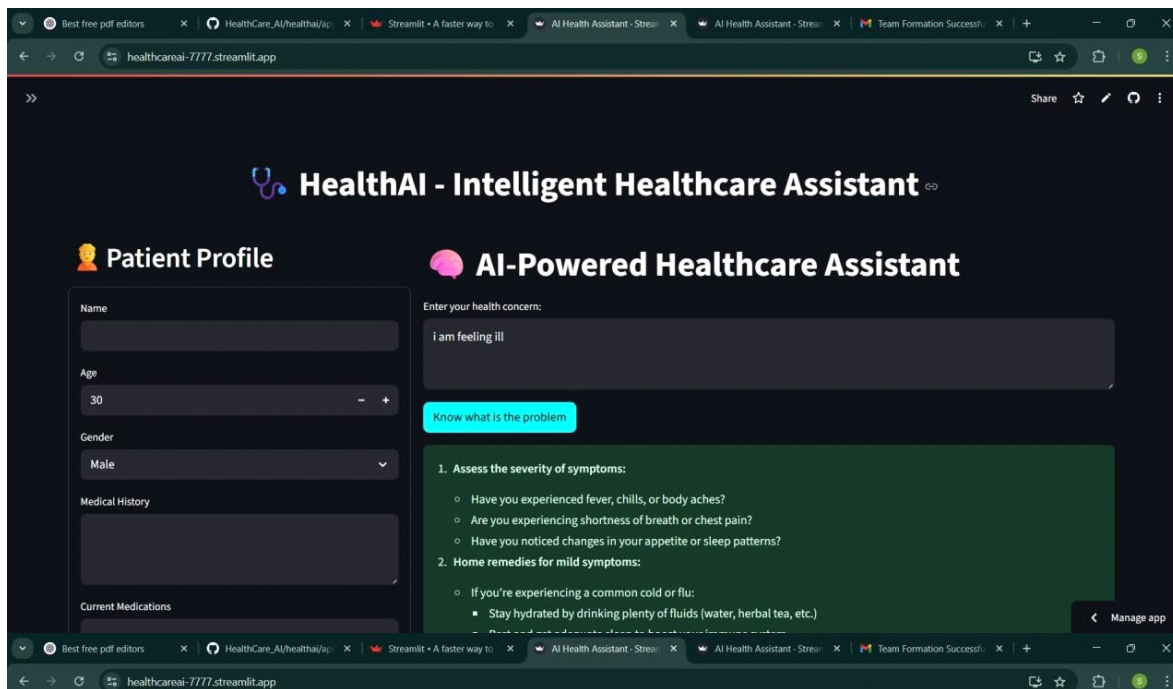
```
File Edit Selection View Go Run Terminal Help
HealthCareAI

health.py
1 # Import necessary libraries
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn.metrics import silhouette_score
6
7 # Set page configuration
8 plt.rcParams['figure.figsize'] = (10, 10)
9
10 # Main function
11 def main():
12     # Create a sample dataset
13     data = pd.DataFrame({
14         'age': [25, 35, 45, 55, 65],
15         'height': [1.7, 1.8, 1.9, 2.0, 2.1],
16         'weight': [70, 80, 90, 100, 110],
17         'blood_pressure': [120, 130, 140, 150, 160],
18         'heart_rate': [70, 80, 90, 100, 110]
19     })
20
21     # Perform K-Means clustering
22     kmeans = KMeans(n_clusters=3, random_state=42)
23     clusters = kmeans.fit_predict(data)
24
25     # Calculate silhouette score
26     silhouette_avg = silhouette_score(data, clusters)
27
28     # Print results
29     print(f"K-Means Clustering Results")
30     print(f"Number of clusters: 3")
31     print(f"Silhouette score: {silhouette_avg}")
32
33     # Plot the results
34     plt.figure(figsize=(10, 10))
35     plt.scatter(data['age'], data['height'], c=clusters, s=100, edgecolor='k')
36     plt.title('K-Means Clustering Results')
37     plt.xlabel('Age')
38     plt.ylabel('Height')
39     plt.show()
40
41 # Call the main function
42 if __name__ == '__main__':
43     main()
44
45 # End of file
46
```

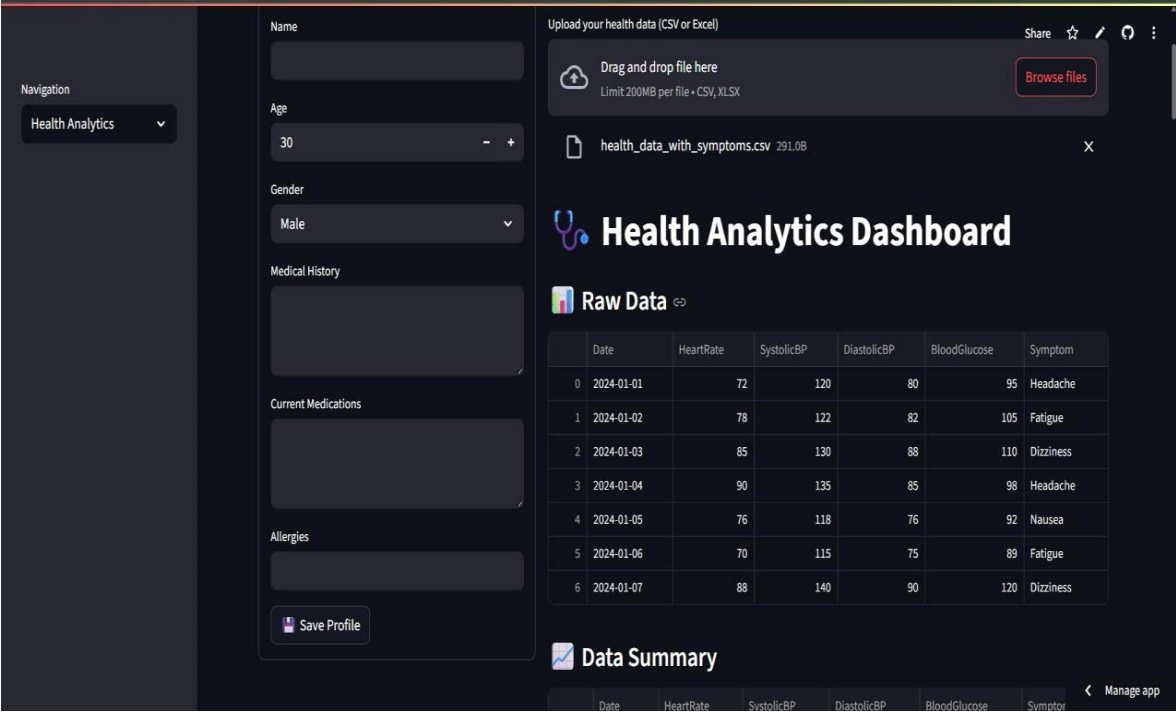
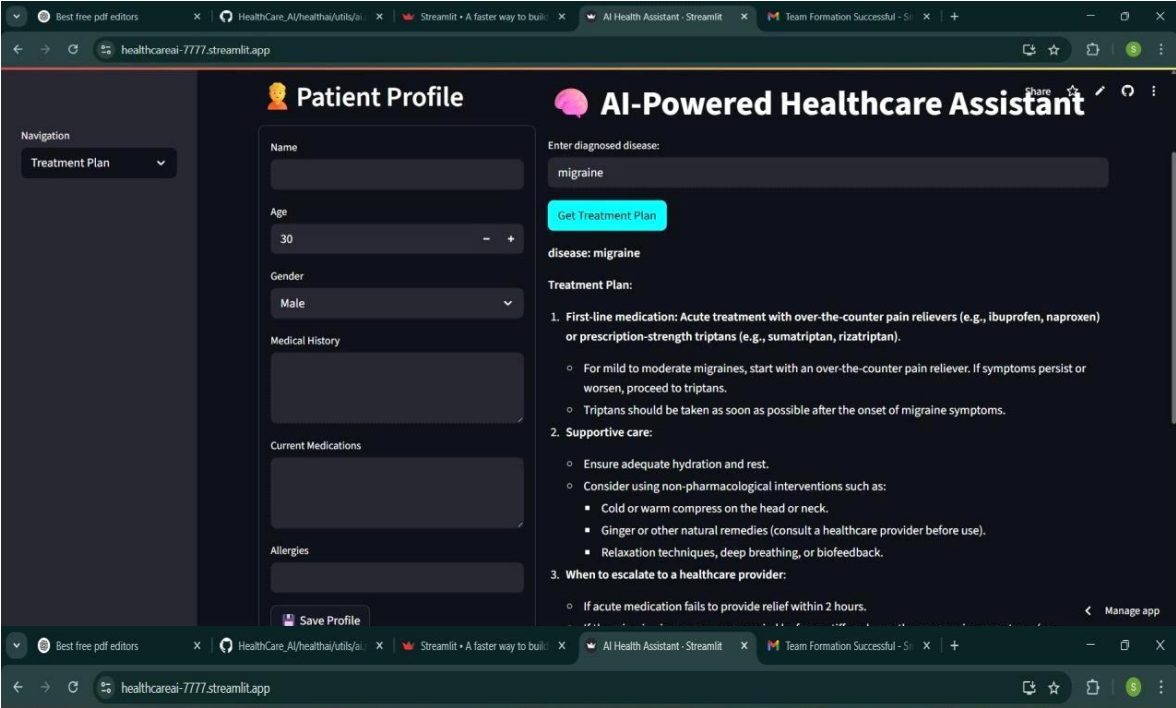
```
File Edit Selection View Go Run Terminal Help
HealthCareAI

display_health_analytics.py
1 # Import necessary libraries
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 from sklearn.cluster import KMeans
5 from sklearn.metrics import silhouette_score
6
7 # Set page configuration
8 plt.rcParams['figure.figsize'] = (10, 10)
9
10 # Main function
11 def main():
12     # Create a sample dataset
13     data = pd.DataFrame({
14         'age': [25, 35, 45, 55, 65],
15         'height': [1.7, 1.8, 1.9, 2.0, 2.1],
16         'weight': [70, 80, 90, 100, 110],
17         'blood_pressure': [120, 130, 140, 150, 160],
18         'heart_rate': [70, 80, 90, 100, 110]
19     })
20
21     # Perform K-Means clustering
22     kmeans = KMeans(n_clusters=3, random_state=42)
23     clusters = kmeans.fit_predict(data)
24
25     # Calculate silhouette score
26     silhouette_avg = silhouette_score(data, clusters)
27
28     # Print results
29     print(f"K-Means Clustering Results")
30     print(f"Number of clusters: 3")
31     print(f"Silhouette score: {silhouette_avg}")
32
33     # Plot the results
34     plt.figure(figsize=(10, 10))
35     plt.scatter(data['age'], data['height'], c=clusters, s=100, edgecolor='k')
36     plt.title('K-Means Clustering Results')
37     plt.xlabel('Age')
38     plt.ylabel('Height')
39     plt.show()
40
41 # Call the main function
42 if __name__ == '__main__':
43     main()
44
45 # End of file
46
```

OUTPUT:









- **Known Issues**

- ☐ Generic model outputs due to lack of medical domain fine-tuning
- ☐ Internet dependency when using IBM watsonx.
- ☐ No data persistence (currently stateless app)

- **Future Enhancements**

- ☒ Add user authentication and patient record storage
- ☒ Multilingual prompt support
- ☒ Mobile version of the app
- ☒ Integrate with real-time health APIs or EHRs